XOR Problem

```
import numpy as np, tensorflow as tf, matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

# XOR dataset
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])

# Build model
model = Sequential([
    Dense(2, input_dim=2, activation='sigmoid', name='hidden'),
    Dense(1, activation='sigmoid', name='output')
])
model.compile(optimizer=Adam(1), loss='binary_crossentropy', metrics=['accuracy'])

# Manual weight initialization
model.layers[0].set_weights([np.array([[2.524,-5.875],[-2.715,5.026]]),
                             np.array([-1.429,-3.095])])
model.layers[1].set_weights([np.array([[1.531],[3.24]]), np.array([-1.099])])

# Early-stop callback
class StopWhenPerfect(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        p = (self.model.predict(X, verbose=0) > 0.5).astype(int)
        print(f"\nEpoch {epoch+1}:", p.ravel())
        if np.array_equal(p, y):
            print("All predictions correct. Stopping early.")
            self.model.stop_training = True

# Train model
history = model.fit(X, y, epochs=100, verbose=0, callbacks=[StopWhenPerfect()])

# Final predictions
pred = (model.predict(X) > 0.5).astype(int)
for i in range(len(X)):
    print(f"Input:{X[i]} -> Pred:{pred[i][0]}")

# Plot accuracy & loss
plt.subplot(1,2,1)
plt.plot(history.history['accuracy']); plt.title("Accuracy"); plt.grid()
plt.subplot(1,2,2)
plt.plot(history.history['loss']); plt.title("Loss"); plt.grid()
plt.show()
```

MNIST

```
import numpy as np, matplotlib.pyplot as plt, tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical

# Load dataset
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalize and one-hot encode
X_train, X_test = X_train/255.0, X_test/255.0
y_train, y_test = to_categorical(y_train), to_categorical(y_test)

# Build model
model = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

# Train
history = model.fit(X_train, y_train, epochs=10, batch_size=64, validation_split=0.2)

# Evaluate
loss, acc = model.evaluate(X_test, y_test)
print(f"\nTest Accuracy: {acc:.4f}, Test Loss: {loss:.4f}")

# Plot accuracy and loss
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], '--', label='Val')
plt.title('Accuracy'); plt.legend(); plt.grid()
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train')
plt.plot(history.history['val_loss'], '--', label='Val')
plt.title('Loss'); plt.legend(); plt.grid()
plt.tight_layout(); plt.show()

# Predict first 10 test images
pred = model.predict(X_test[:10])
for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(X_test[i], cmap='gray')
    plt.title(f"P:{np.argmax(pred[i])}, T:{np.argmax(y_test[i])}")
    plt.axis('off')
plt.show()
```

CHARACTER RECOGNITION

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
Flatten, Dense, Dropout
import matplotlib.pyplot as plt

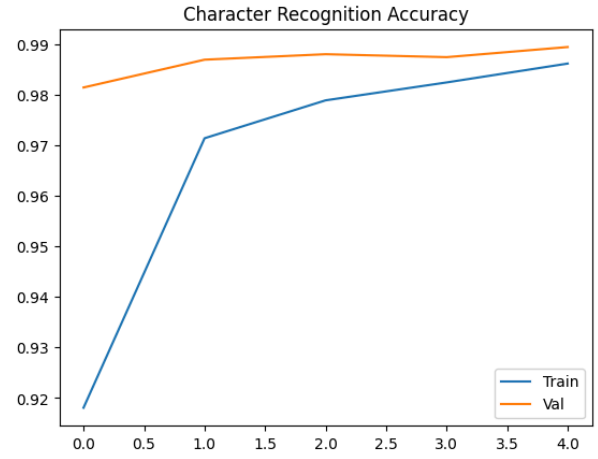
# Load dataset (use LFW, CelebA, or custom faces dataset)
(X_train, y_train), (X_test, y_test) =
tf.keras.datasets.cifar10.load_data()
X_train, X_test = X_train / 255.0, X_test / 255.0

# Build CNN (same structure as Character Recognition)
model = Sequential([
    Conv2D(32, (3,3), activation='relu',
input_shape=(32,32,3)),
    MaxPooling2D((2,2)),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')  # 10 classes for CIFAR-10
])

# Compile & Train
model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
history = model.fit(X_train, y_train, epochs=5,
validation_split=0.2)

# Evaluate
print("Test Accuracy:", model.evaluate(X_test, y_test)[1])

# Plot
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Val')
plt.title("Face Recognition Accuracy")
plt.legend()
plt.show()
```



FACE RECOG

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
Flatten, Dense, Dropout
import matplotlib.pyplot as plt

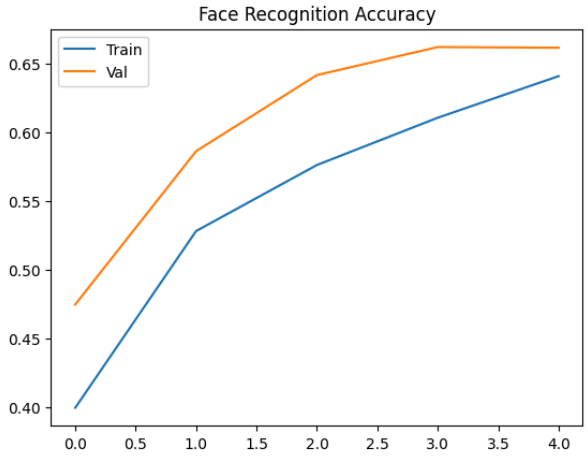
# Load dataset (use LFW, CelebA, or custom faces dataset)
(X_train, y_train), (X_test, y_test) =
tf.keras.datasets.cifar10.load_data()
X_train, X_test = X_train / 255.0, X_test / 255.0

# Build CNN (same structure as Character Recognition)
model = Sequential([
    Conv2D(32, (3,3), activation='relu',
input_shape=(32,32,3)),
    MaxPooling2D((2,2)),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')  # 10 classes for CIFAR-10
])

# Compile & Train
model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
history = model.fit(X_train, y_train, epochs=5,
validation_split=0.2)

# Evaluate
print("Test Accuracy:", model.evaluate(X_test, y_test)[1])

# Plot
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Val')
plt.title("Face Recognition Accuracy")
plt.legend()
plt.show()
```



LANGUAGE MODELING

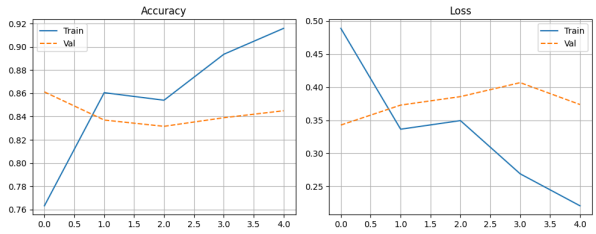
```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import
pad_sequences
# 1. Load and preprocess dataset
with open('dataset1.txt', 'r', encoding='utf-8') as f:
    text = f.read().lower()
tokenizer = Tokenizer()
tokenizer.fit_on_texts([text])
total_words = len(tokenizer.word_index) + 1
# 2. Create input sequences
sequences = []
for line in text.split('\n'):
    tokens = tokenizer.texts_to_sequences([line])[0]
    for i in range(2, len(tokens) + 1):
        sequences.append(tokens[:i])
max_len = max(len(s) for s in sequences)
sequences = pad_sequences(sequences, maxlen=max_len,
padding='pre')
X, y = sequences[:, :-1],
tf.keras.utils.to_categorical(sequences[:, -1],
num_classes=total_words)
# 3. Build RNN model
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(total_words, 100,
input_length=max_len - 1),
    tf.keras.layers.SimpleRNN(128),
    tf.keras.layers.Dense(total_words, activation='softmax')
])
model.compile(loss='categorical_crossentropy',
optimizer='adam', metrics=['accuracy'])
model.summary()
# 4. Train
history = model.fit(X, y, epochs=50, batch_size=64, verbose=1)
# 5. Text generation
def generate_text(seed_text, next_words=20):
    for _ in range(next_words):
        token_list =
tokenizer.texts_to_sequences([seed_text])[0]
        token_list = pad_sequences([token_list],
maxlen=max_len - 1, padding='pre')
        pred = np.argmax(model.predict(token_list, verbose=0))
        word = tokenizer.index_word.get(pred, '')
        seed_text += ' ' + word
        if word.endswith('.'): break
    return seed_text

print("\nGenerated text:\n", generate_text("deep learning
has", 20))
# 6. Plot accuracy & loss
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'])
plt.title('Training Accuracy'); plt.grid()
plt.subplot(1,2,2)
plt.plot(history.history['loss'])
plt.title('Training Loss'); plt.grid()
plt.tight_layout(); plt.show()
```

LSTM

```
import numpy as np, matplotlib.pyplot as plt, tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import
pad_sequences
# 1. Load IMDb dataset
max_words, max_len = 10000, 200
(X_train, y_train), (X_test, y_test) =
imdb.load_data(num_words=max_words)
# 2. Pad sequences
X_train = pad_sequences(X_train, maxlen=max_len)
X_test = pad_sequences(X_test, maxlen=max_len)
# 3. Build model
model = Sequential([
    Embedding(max_words, 128, input_length=max_len),
    LSTM(128, dropout=0.2, recurrent_dropout=0.2),
    Dense(1, activation='sigmoid')
])
model.compile(loss='binary_crossentropy', optimizer='adam',
metrics=['accuracy'])
model.summary()
# 4. Train model
history = model.fit(X_train, y_train,
                    epochs=5, batch_size=128,
                    validation_split=0.2, verbose=1)
# 5. Evaluate
loss, acc = model.evaluate(X_test, y_test)
print(f"\nTest Accuracy: {acc*100:.2f}%, Test Loss:
{loss:.4f}")
# 6. Plot accuracy and loss
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], '--', label='Val')
plt.title('Accuracy'); plt.legend(); plt.grid()
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train')
plt.plot(history.history['val_loss'], '--', label='Val')
plt.title('Loss'); plt.legend(); plt.grid()
plt.tight_layout(); plt.show()
# 7. Predict sample review
word_index = imdb.get_word_index()
reverse_index = {v: k for k, v in word_index.items()}
def decode_review(encoded):
    return ' '.join([reverse_index.get(i-3, '?') for i in
encoded])

sample = X_test[0].reshape(1, -1)
pred = model.predict(sample)[0][0]
print("\nSample Review:\n", decode_review(X_test[0]))
print("Predicted Sentiment:", "Positive" if pred > 0.5 else
"Negative")
```



PARTS OF SPEECH

```
Import tensorflow as tf, numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical

# 1. Sample data
sentences = ["I love coding", "He plays football", "She reads books"]
tags = ["PRON VERB NOUN", "PRON VERB NOUN", "PRON VERB NOUN"]

# 2. Tokenize and pad
w_tok, t_tok = Tokenizer(), Tokenizer()
w_tok.fit_on_texts(sentences); t_tok.fit_on_texts(tags)
X = pad_sequences(w_tok.texts_to_sequences(sentences), padding='post')
y = pad_sequences(t_tok.texts_to_sequences(tags), padding='post')
y = to_categorical(y, num_classes=len(t_tok.word_index)+1)

# 3. Build model
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(len(w_tok.word_index)+1, 64,
input_length=X.shape[1]),
    tf.keras.layers.LSTM(64, return_sequences=True),
    tf.keras.layers.Dense(len(t_tok.word_index)+1, activation='softmax')
])
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])

# 4. Train & Evaluate
hist = model.fit(X, y, epochs=30, verbose=1)
print("Test Accuracy:", model.evaluate(X, y, verbose=0)[1])

# 5. Predict POS tags
i2t = {v: k for k, v in t_tok.word_index.items()}
def predict(sentence):
    seq = pad_sequences(w_tok.texts_to_sequences([sentence]),
maxlen=X.shape[1], padding='post')
    pred = np.argmax(model.predict(seq, verbose=0), axis=-1)[0]
    return list(zip(sentence.split(), [i2t.get(p, '-') for p in pred[:len(sentence.split())]]))

print("\nPredicted Tags:", predict("She loves coding"))
```

```
Test Accuracy: 0.5555555820465088

Predicted Tags: [('She', 'pron'), ('loves', 'noun'), ('coding',
'noun')]
```

TRANSLATION

```
import tensorflow as tf, numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import
pad_sequences   # ✅ FIXED
from tensorflow.keras.utils import to_categorical
# 1. Tiny dataset
eng = ["I love cats", "He is playing", "She eats rice"]
tam = ["நான் பூனைகளை விரும்புகிறேன்", "அவன்
விளையாடுகிறான்", "அவள் அரிசி சாப்பிடுகிறாள்"]
# 2. Tokenize & pad
etok, ttok = Tokenizer(), Tokenizer()
etok.fit_on_texts(eng); ttok.fit_on_texts(tam)
X = pad_sequences(etok.texts_to_sequences(eng),
padding='post')
y = pad_sequences(ttok.texts_to_sequences(tam),
padding='post')
y = to_categorical(y, len(ttok.word_index)+1)
# 3. Simple Encoder-Decoder model
m = tf.keras.Sequential([
    tf.keras.layers.Embedding(len(etok.word_index)+1, 64,
input_length=X.shape[1]),
    tf.keras.layers.LSTM(64, return_sequences=True),
    tf.keras.layers.Dense(len(ttok.word_index)+1,
activation='softmax')
])
m.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
m.fit(X, y, epochs=50, verbose=1)
# 4. Translate function
idx2word = {v: k for k, v in ttok.word_index.items()}
def translate(text):
    seq = pad_sequences(etok.texts_to_sequences([text]),
maxlen=X.shape[1], padding='post')
    pred = np.argmax(m.predict(seq), axis=-1)[0]
    return ' '.join([idx2word.get(i, '') for i in
pred]).strip()

print("\nEnglish: I love cats")
print("Predicted Tamil:", translate("I love cats"))
```

```
English: I love cats
1/1
```

```
0s 205ms/step
Predicted Tamil: பூனைகளை
பூனைகளை விரும்புகிறேன்
```

GANS

```
import tensorflow as tf, numpy as np, matplotlib.pyplot as plt
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Reshape, Flatten,
LeakyReLU
from tensorflow.keras.optimizers import Adam
# Load & normalize MNIST
(X, _), _ = tf.keras.datasets.mnist.load_data()
X = (X.astype('float32') - 127.5) / 127.5
X = X.reshape(-1, 28, 28)   # ✅ keep image shape
latent_dim = 100# Generator
G = Sequential([
    Dense(128, activation='relu', input_dim=latent_dim),
    Dense(784, activation='tanh'),
    Reshape((28, 28))
])# Discriminator
D = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(1, activation='sigmoid')
])
D.compile(optimizer=Adam(0.0002), loss='binary_crossentropy')
# GAND.trainable = False
GAN = Sequential([G, D])
GAN.compile(optimizer=Adam(0.0002),
loss='binary_crossentropy')
# Train
for e in range(1001):
    noise = np.random.normal(0, 1, (64, latent_dim))
    fake = G.predict(noise, verbose=0)
    idx = np.random.randint(0, X.shape[0], 64)
    real = X[idx]                    # ✅ correct shape now
    D.train_on_batch(real, np.ones((64, 1)))
    D.train_on_batch(fake, np.zeros((64, 1)))
    GAN.train_on_batch(noise, np.ones((64, 1)))
    if e % 200 == 0:
        imgs = 0.5 * G.predict(np.random.normal(0, 1, (25,
latent_dim)), verbose=0) + 0.5
        plt.figure(figsize=(4,4))
        for i in range(25):
            plt.subplot(5,5,i+1)
            plt.imshow(imgs[i], cmap='gray')
            plt.axis('off')
        plt.suptitle(f"Epoch {e}")
        plt.show()
```

